

Principles of Transaction-Oriented Database Recovery.

Haeder83: Theo Haerder, Andreas Reuter,
ACM Computing Surveys,
vol 15, no 4, Dec 1983.

Transactions

- Multiple users manipulating data safely
- ACID properties of a transaction
 - Atomicity: transform is all or nothing
 - Consistency: make only correct changes
 - Isolation: partial changes hidden from others
 - Durability: changes survive subsequent failures
- AID provided by DBMS, C by programmer

Failures

- Transaction failure
 - Code aborts, based on input/database inconsistency [programmer escapes complexity]
 - Mechanical aborts caused by concurrency control solutions to isolation
 - Frequent events, “instant” recovery needed
- System failure (fail-stop)
 - DBMS bug, OS fault, HW failure: wipe out volatile memory but durable memory (disk) survives
 - Infrequent events, “minutes” to recover
- Media failure (fail-stop)
 - IO code bugs, disk HW failures: lose of disk info
 - Rare events, “hours” to recover from checkpoints & audit logs

Recoverable System Model

- UNDO: rollback aborted transaction
 - Transaction (transaction failure) or global (system failure)
- REDO: repeat transaction on old DB data
 - Partial (system failure) or global (media failure)

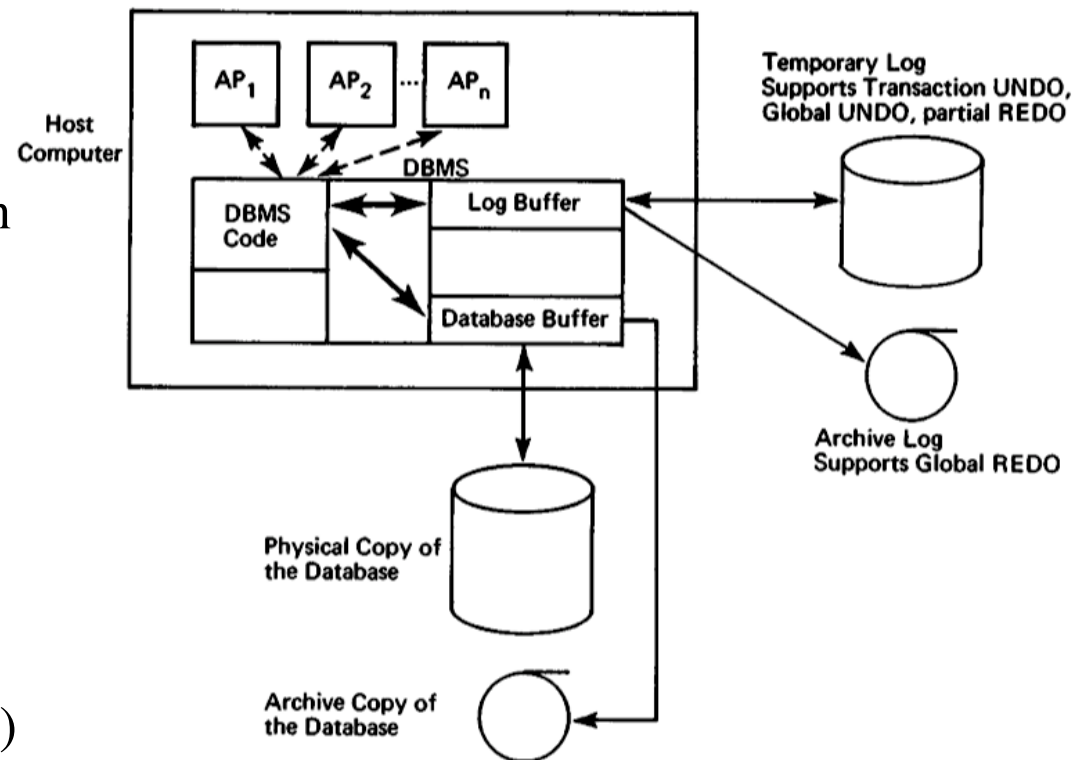


Figure 4. Storage hierarchy of a DBMS during normal mode of operation.

Views

- Current DB: on disk + memory buffers
 - Some transactions are in flight with changes in buffers that might not occur
- Materialized DB: on crash restart before apply log processing
 - Some completed changes may not be visible because the buffer was lost
- Physical DB: on disk
 - All blocks including out-of-date blocks & possibly incomplete data structures

Sequencing Views

- Changing non-volatile memory
 - “Modifications” of current DB may cause “writes” to physical DB that are not part of materialized DB until pointer structs are updated, ie. “propogated” (like LFS updates, which learned from DB theory)
 - Some DBs overwrite prior copies so write=propagate, but this makes changes in materialized DB non-atomic (harder to recover)

Temporary Log Files

- Redundant information for coping with failure
 - On-disk temporary (write-ahead) log file contains all that is needed to transform materialized DB to current DB
 - Memory pressure can push uncommitted dirty data to database; in “overwrite” DBs this requires UNDO log records (STEAL) written before propagation; in non-overwrite DBs such writes are “forgotten” when memory is lost
 - Commit logically forces propagation (FORCE), but efficiency concerns cause DBs to avoid synchronous IO, instead writing REDO log records before transaction commit
- Log types
 - Physical vs Logical: capture data values or operations giving values
 - State or transition: capture full values or differences from last values
 - Page or record: capture full page values or only the records changing
 - Tradeoffs involve simplicity/speed of recovery vs density of log

Crash Recovery

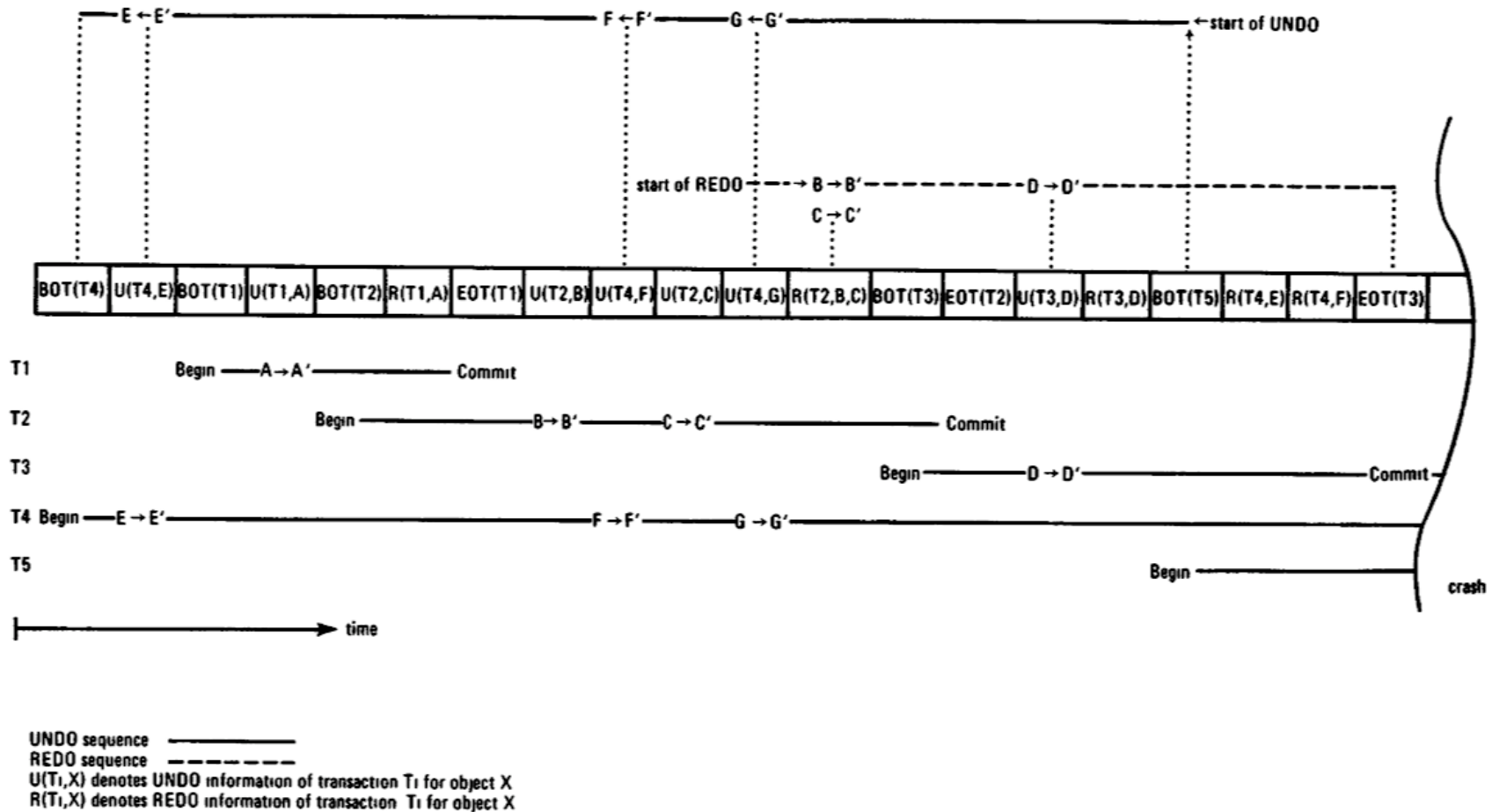


Figure 8. A crash recovery scenario.

Checkpoint

- Checkpoints limit REDO processing
 - REDO basically starts at beginning of log, but that can be really slow
 - FORCE propagation means no REDO (just UNDOs)
 - Quiesce all transactions, propagate all dirty data, write log entry (transaction consistent)
 - Allows partial REDO to start here and global UNDO recovery processing to stop here
 - Otherwise UNDO needs to find BEGIN for oldest incomplete transaction
 - Quiesce transaction-caused actions (calls into DBMS), propagate all dirty data, write log entry (action consistent)
 - Allows partial REDO recovery processing to start here
 - Fuzzy checkpoints are those that “propagate” only to log, to reduce REDO processing on restart

Comparisons

Table 4. Evaluation of Logging and Recovery Techniques Based on the Introduced Taxonomy

propagation strategy	¬ATOMIC							ATOMIC				
buffer replacement	STEAL				¬STEAL			STEAL			¬STEAL	
EOT processing	FORCE	¬FORCE			FORCE	¬FORCE		FORCE	¬FORCE		FORCE	¬FORCE
checkpoint type	TOC	TCC	ACC	FUZZY	TOC	TCC	FUZZY	TOC	TCC	ACC	TOC	TCC
materialized DB state after system failure	DC	DC	DC	DC	DC	DC	DC	TC	TC	AC	TC	TC
cost of transaction UNDO	+	+	+	+	--	--	--	-	-	+	--	--
cost of partial REDO at restart	--	-	-	+	--	-	-	--	-	-	--	-
cost of global UNDO at restart	+	+	+	+	--	--	--	--	--	+	--	--
overhead during normal processing	--	--	--	--	--	--	--	+	+	+	+	+
frequency of check-points	+	-	-	-	+	-	-	+	-	-	+	-
checkpoint cost	+	++	++	-	+	++	+	+	++	++	+	++

Notes:

Abbreviations: DC, device consistent (chaotic); AC, action consistent; TC, Transaction consistent.
Evaluation symbols: --, very low; -, low; +, high; ++, very high.